

ZeroID — Engineering Status Report (2026-06-29)

Contents

ZeroID — Engineering Status Report	1
1. Executive summary	1
2. What ZeroID is (positioning)	2
3. Architecture overview	2
4. Technology stack	3
5. What's built — by domain	4
5.1 Aethelred conformance boundary (W1–W6) — <code>src/lib/aethelred/</code> (16 modules, 15 test suites)	4
5.2 Phase 2 — the moat	4
5.3 AI Agent Passport v1 (flagship)	4
5.4 Smart contracts (14) — <code>contracts/</code> , Foundry tests in <code>test/foundry/</code>	4
5.5 Economic flywheel — <code>FeeRouter.sol</code>	5
5.6 Partner integration surface — <code>/api/v1/partners/*</code>	5
5.7 Backend platform (the rest of the 38 services)	5
5.8 Cross-cutting hardening (most recent work)	5
5.9 Frontend (19 pages)	5
6. Testing & quality	5
7. Security & compliance posture	6
8. Honest gaps — what is NOT done (and why)	6
Infra-gated (code-complete behind flags/adapters; flip after end-to-end verification)	6
Genuine work remaining	7
Repo / integration hygiene (recommend a decision)	7
9. Suggested questions for the consultant	7

ZeroID — Engineering Status Report

Date: 2026-06-29 **Audience:** External consultant review **Repo:** `zeroid` (local: `~/Downloads/zeroid`)
Active branch: `feat/economic-flywheel` (latest; stacked on `feat/ai-agent-passport-v1` → `design/aethelred-conformance-moat`)

Purpose: a complete, honest picture of what ZeroID is today — architecture, technology, code-base, test posture, security/compliance, and the gaps that are genuinely blocked on infrastructure vs. those that are code-complete. Intended to let you advise on **where to focus next and what to fix**.

1. Executive summary

ZeroID is a **privacy-preserving decentralized identity (DID/VC) platform** positioned as:

1. “**The AI agent’s passport**” — the flagship v1 scenario: an AI agent proves, in zero-knowledge, that *its human controller* is eligible for a regulated action, under a named policy, with a full audit trail —

without exposing the controller's PII.

2. **The identity spine of the Aethelred ecosystem** — Wallet (custody) and Cruzible (staking) call ZeroID for eligibility, agent compliance scans, and warrant-bound conditional disclosure.

It is built to **conform to the Aethelred L1** canonical stack (not a generic EVM): ZK precompiles, TEE attestation, post-quantum signing, and EVM-compatible Solidity, isolated behind a single conformance boundary.

Maturity at a glance:

Area	State
Core platform (identity, credentials, verification, policy governance, compliance)	Built & tested
AI Agent Passport v1 (backend + frontend, end-to-end)	Built & tested
Smart contracts (14) + economic flywheel + conditional disclosure quorum	Built & tested (incl. fuzz + invariant)
Aethelred conformance boundary (ZK / TEE / PQC / hooks)	Built & tested behind adapters; real providers are flag-gated
Partner surface (Wallet / Cruzible endpoints)	Built & tested (DI; idempotent)
Cross-cutting hardening (idempotency, unified errors, security docs)	Built & tested
Testnet activation, DB apply, deployed addresses, real TEE/EZKL/PQC backends	Infra-gated — not done (see §8)

Code volume (excl. dependencies): backend ~54k LOC · frontend ~88k LOC (incl. conformance lib ~2k) · Solidity contracts ~9.4k LOC.

2. What ZeroID is (positioning)

- **Subject of trust:** a human (or organization) holds a DID and verifiable credentials; an AI agent is a *delegated* actor bound to a controller DID with scoped capabilities and a risk ceiling.
- **The unit of value:** an **eligibility proof** — “this subject satisfies policy P (age / jurisdiction / sanctions / risk / credential validity / non-revocation)” — produced as a zero-knowledge proof and recorded as a tamper-evident decision receipt.
- **The differentiator vs. incumbents:** ZK + TEE + post-quantum + on-chain conditional disclosure (regulator-grade, warrant-bound, quorum-gated reveal) on a sovereign L1 — rather than a centralized KYC API.

3. Architecture overview

Four layers, with the Aethelred dependency isolated to one seam:

```
+-----+
| FRONTEND - Next.js 14 / React / React Query / viem + wagmi |
| 19 pages: identity, credentials, verification,                |
| agent-identity, eligibility, compliance, governance,          |
| marketplace, ...                                              |
+-----+
| REST /api/v1/*
+-----+
```

BACKEND - Express + TypeScript + Prisma (Postgres)	
38 service modules 11 route groups 32 Prisma models	
identity credential verification (eligibility)	
policy governance compliance AI (risk/fraud/bio)	
enterprise (OIDC, issuer-trust, SLA, webhooks) partners	
+-----+-----+	
single conformance seam	
+-----v-----+	
src/lib/aethelred/ - Aethelred conformance boundary (16)	
zk verify encoding vkeys attestation (TEE/DCAP)	
seals signing (PQC hybrid) liveness (EZKL)	
disclosure shamir client react hooks	
[CI guard: no Aethelred SDK imports anywhere else]	
+-----+-----+	
EVM + precompiles	
+-----v-----+	
CONTRACTS (Foundry / Solidity ^0.8.20) - 14 contracts	
ZeroID CredentialRegistry ZKCredentialVerifier	
SelectiveDisclosure AccumulatorRevocation BBSPlus	
ThresholdCredential TEEAttestationRegistry AIAgentReg	
CrossChainIdentityBridge GovernanceModule	
RegulatoryCompliance ConditionalDisclosure FeeRouter	
+-----+-----+	
deployed to -> Aethelred L1	

The **strangler-fig boundary** (`src/lib/aethelred/`) is the most important architectural decision: every Aethelred-specific concern (proof byte formats, vkey registry, TEE quote verification, PQC signing, React hooks) lives behind this one directory. A **CI guard** forbids importing the Aethelred SDK anywhere else, so the rest of ZeroID is insulated from chain-side churn and the integration can be activated gate-by-gate.

4. Technology stack

Layer	Technology
Frontend	Next.js 14 (App Router), React, TanStack React Query, viem + wagmi, TypeScript
Backend	Node + Express, TypeScript, Prisma ORM → PostgreSQL, Zod validation, JWT auth + RBAC
Contracts	Solidity ^0.8.20, Foundry (forge), OpenZeppelin (AccessControl, Pausable, ReentrancyGuard)
Crypto	Groth16 (BN254), Shamir Secret Sharing over GF(256), AES-256-GCM, SHA-256 commitments; PQC hybrid
Testing	ML-DSA-65 + ECDSA (ML-KEM-768 KEM) — via adapters Jest (backend + frontend), Foundry (unit + fuzz + invariant), TDD throughout
Target L1 (Aethelred)	Cosmos-SDK v0.50 / CometBFT; Rust VM with ZK precompiles (Groth16 / PLONK / EZKL over BN254 / BLS12-381); DCAP TEE attestation; PQC (ML-DSA-65 hybrid, ML-KEM-768); EVM-compatible; chain IDs 8821 / 88210

5. What’s built — by domain

5.1 Aethelred conformance boundary (W1–W6) — `src/lib/aethelred/` (16 modules, 15 test suites)

Delivered as six increments behind the single seam:

- **W1 — boundary scaffold + CI guard:** `index.ts`, `client.ts`; guard enforcing no `@aethelred/sdk` imports outside the boundary.
- **W2 — ZK verification cutover:** `zk.ts` (Groth16 verify), `verify.ts` (verification strategy), `encoding.ts` (snarkjs→arkworks byte format: G2 limb order / compression), `vkeys.ts` (verification-key registry by circuit id). Gated by `NEXT_PUBLIC_CANONICAL_VERIFY`.
- **W3 — TEE → DCAP attestation + Digital Seals:** `attestation.ts` (DCAP quote verification), `seals.ts` (Digital Seals binding).
- **W4 — PQC hybrid signing:** `signing.ts` (ML-DSA-65 + ECDSA hybrid). Gated by `NEXT_PUBLIC_PQC_SIGNING`.
- **W5 — React hooks:** `react.ts` (`@aethelred/sdk/react-style` hooks for the frontend).
- **W6 — ecosystem compatibility CI:** boundary guard + workflow.

5.2 Phase 2 — the moat

- **EZKL zkML liveness** (`liveness.ts`): verifiable on-device liveness via an EZKL circuit (model→ONNX→circuit pipeline pending; verifier path built).
- **Conditional disclosure** (`disclosure.ts`, `disclose.ts`, `disclosure-contract.ts`, `shamir.ts`): regulator-grade, warrant-bound reveal. Shamir key-split escrow over GF(256), AES-256-GCM payload, SHA-256 commitment, key-shred erasure (right-to-be-forgotten). On-chain quorum via `ConditionalDisclosure.sol` with a **governance-race fix** (threshold snapshotted at request time) and `pause()`.

5.3 AI Agent Passport v1 (flagship)

- **Backend:** `agent-passport.ts` (scope vocabulary `eligibility.read / audit.read / identity.read`; pure policy `POLICY_AGENT_ELIGIBILITY_VIEW_V1` with 6 conditions and explicit deny codes); `agent-eligibility.ts` (DI orchestration; agent→controller binding, scope + risk-tier checks, recorded `AgentAction`); route `POST /api/v1/ai/agents/eligibility/proof`.
- **Key design:** the agent path **reuses the exact human eligibility handler** (`eligibilityProofHandler`, extracted from `verification.ts`) in-process — no re-implementation, behaviour-identical.
- **Frontend:** agent registry client + hooks + `agent-identity` page (register wizard wired to real v1 registration: name + scopes + risk tier); audit timeline filter + “AI Agent X acting for Y” attribution.
- **Schema:** additive (AI Agent + controllerDid/riskTier; AgentCredential + scopes/maxRiskTier/controllerDid/revocation; AgentAction + controllerDid/policyId/decision).

5.4 Smart contracts (14) — `contracts/`, Foundry tests in `test/foundry/`

ZeroID, CredentialRegistry, ZKCredentialVerifier, SelectiveDisclosure, AccumulatorRevocation, BBSPlusCredential, ThresholdCredential, TEEAttestationRegistry, AI Agent Registry, CrossChainIdentityBridge, GovernanceModule, RegulatoryCompliance, ConditionalDisclosure, FeeRouter (+ BN254 library + interfaces/verifiers).

Production hardening on the economic + disclosure contracts: OpenZeppelin AccessControl / Pausable / ReentrancyGuard, CEI ordering, custom errors. **Fuzz + invariant** suites for `FeeRouter`, `ConditionalDisclosure`, and `AccumulatorRevocation` (e.g. value-conservation invariant on the fee router; quorum-authorization invariant on disclosure).

5.5 Economic flywheel — FeeRouter.sol

Per-operation fee → configurable **burn** + **Cruzible** staking-sink split (CEI, pausable, role-gated, with totalBurned/totalToCruzible/totalRouted accounting). Deployed via `script/Deploy.s.sol` (env-driven; testnet + mainnet from one script).

5.6 Partner integration surface — /api/v1/partners/*

DI orchestrators (`partner-service.ts`) that **reuse** existing ZeroID logic: - `POST /partners/wallet/eligibility`
· `POST /partners/wallet/disclosure` · `GET /partners/wallet/evidence/:id` - `POST /partners/cruzible/pools/:id`
· `POST /partners/cruzible/pools/:id/agent-scan`

All POSTs are **idempotent** (Idempotency-Key header) and use the unified error taxonomy.

5.7 Backend platform (the rest of the 38 services)

- **Identity & credentials:** `identity.ts`, `credential.ts`, DID lifecycle, credential issuance/verification, revocation.
- **Verification / eligibility:** `verification.ts` — the rich eligibility-proof engine (ZK proof verification with replay/nonce/context-binding checks, ~30 typed failure codes, audience/expiry/claims-hash validation). This is the platform's core.
- **Enterprise policy governance:** a full policy decision system — `policy-registry-service`, `policy-definition` / `policy-execution` / `policy-exception` / `policy-receipt` / `policy-governance` / `policy-context` services, backed by a `PolicyDecisionLedger` with integrity hashing and approval trails.
- **Compliance:** `jurisdiction-engine`, `sanctions-screening`, `regulatory-reporting`, `data-sovereignty`.
- **AI:** `risk-scoring`, `fraud-detection`, `behavioral-biometrics`, `compliance-advisor`.
- **Enterprise integration:** `oidc-bridge` + `oidc-claims`, `issuer-trust-service`, `organization-service`, `sla-monitor`, `webhook-system`, `api-gateway`, `enterprise-key-signer`, `regulatory-submission-signing`.
- **Infrastructure:** `tee.ts`, `zkproof.ts`, `government-api.ts`, `audit-integrity.ts`, `circuit-artifacts.ts`, `production-safety.ts`, `idempotency.ts`, `errors.ts`.

5.8 Cross-cutting hardening (most recent work)

- **Idempotency** (`services/idempotency.ts`): generic `IdempotencyStore<T>` + Prisma store + `withIdempotency` helper + Idempotency-Key normalizer; backs the agent + all partner POSTs (operation-scoped keys, first-write-wins, `idempotency_records` table). Documented as memoization for sequential retries (not a distributed lock).
- **Unified error taxonomy** (`services/errors.ts`): one `ServiceError` base, one `sendServiceError` mapper, stable `{ error, message }` envelope, documented `ServiceErrorCode` vocabulary with passthrough for the eligibility handler's own codes. `AgentEligibilityError` + `PartnerError` extend it.

5.9 Frontend (19 pages)

`identity`, `credentials`, `verification`, `agent-identity`, `eligibility`, `ai-compliance`, `analytics`, `audit`, `cross-chain`, `enterprise`, `governance`, `integrations`, `marketplace`, `regulatory`, `revocation`, `settings`, `admin`, `identity/uae-pass/callback`, `home`.

6. Testing & quality

Suite	Files	Tests	Status
Backend (Jest)	70	746 (~729 it blocks)	743 pass / 3 fail in 2 pre-existing suites (see §8)
Frontend (Jest)	116	~ 2,384 it blocks	green (last full run)
Conformance boundary	15	~62 it blocks	green
Contracts (Foundry)	19 suites	511 test fns (incl. 7 fuzz + 4 invariant)	green at last full run

- **Discipline:** TDD throughout; dependency injection so services are unit-testable without a DB (route tests mock the service layer).
- **Contract rigor:** beyond unit tests, **property-based fuzz** and **stateful invariant** suites on the value-bearing contracts.
- **Boundary guard:** CI fails if the Aethelred SDK leaks outside `src/lib/aethelred/`.
- **Type safety:** `tsc --noEmit` clean across backend.

7. Security & compliance posture

Documented under `docs/`:

- **Threat model v1** (`docs/security/threat-model-v1.md`), **key custody** (`docs/security/key-custody.md`), **incident-response runbook** (`docs/security/incident-response-runbook.md`).
- **DPIA v1** (`docs/compliance/dpia-v1.md`) and **conditional-disclosure regulatory memo** mapping to **FATF / ADGM / GDPR** (warrant-bound reveal, key-shred erasure for right-to-be-forgotten).
- **v1 readiness gate** (`docs/production/zeroid-v1-readiness-gate.md`), **activation runbook** (`docs/ecosystem/activation-runbook.md`), **90-day ecosystem plan** (`docs/ecosystem/90-day-plan-and-integration.md`).
- **Policies:** `eligibility_v1.md`, `agent_identity_v1.md`; **ZK ceremony:** `docs/zk/eligibility-artifact-ceremony.md`.
- **Contract incident response:** every value/disclosure contract ships `pause()`.

8. Honest gaps — what is NOT done (and why)

These are deliberately separated into **infra-gated** (code is ready; needs infrastructure to activate) vs **genuine work remaining**.

Infra-gated (code-complete behind flags/adapters; flip after end-to-end verification)

Gate	What's needed
W2c — ZK verify	Produce a Groth16 proof from a ZeroID circuit, verify via the chain precompile, confirm <code>snarkjs</code> → <code>arkworks</code> byte format; register vkeys; set <code>NEXT_PUBLIC_CANONICAL_VERIFY=true</code> .
W3c — DCAP TEE	A TEE worker emitting a real quote; wire <code>attestation.ts</code> call sites.
W4c — PQC	Inject a real ML-DSA-65 provider; set <code>NEXT_PUBLIC_PQC_SIGNING=true</code> .
Phase 2b — zkML	Train liveness model → ONNX → EZKL circuit/keys; register the circuit.
DB migration apply	Schema is additive and <code>prisma generate</code> is done; <code>prisma migrate deploy</code> not yet run (needs Postgres). Two additive migrations: AI Agent Passport v1, idempotency v1 (tracked SQL copies in <code>docs/ecosystem/</code>).
Deployed addresses	<code>Deploy.s.sol</code> compiles; not broadcast to testnet → no <code>FeeRouter/ConditionalDisclosure</code> addresses in env yet.

Genuine work remaining

- **Real TEE / EZKL / PQC backends** — currently adapters + interfaces + flags; the real provider implementations are pending (these need the chain-side infra to exist).
- **Cruzible / Wallet repo-side wiring** — ZeroID exposes the partner endpoints; the counterpart services and boundary replication into those repos are not built here.
- **Agent-token auth** — v1 uses the controller's JWT (sufficient for pilot); a dedicated agent credential/token flow is a follow-up.
- **Two pre-existing failing backend suites** (independent of recent work, verified by stash isolation):
 - `circuit-artifacts.test.ts` — fails on a missing `circuits/manifest` directory (path/env).
 - `enterprise-compliance-receipts.test.ts` — suite fails to load at `src/routes/enterprise/compliance.ts:1`. These should be triaged: fix or quarantine.

Repo / integration hygiene (recommend a decision)

- The build is spread across **stacked feature branches** not yet merged: `feat/economic-flywheel` → `feat/ai-agent-passport-v1` → `design/aethelred-conformance-moat`, plus several `fix/*` branches; `main` is **~223 commits behind**. There is also a large uncommitted WIP working tree on the active branch. **A consolidation / merge-to-main strategy is needed** before this is reviewable as a single artifact or deployable.

9. Suggested questions for the consultant

1. **Branch consolidation:** what's the right path to a single integratable `main` given the stacked feature branches and the WIP working tree?
2. **Pilot critical path:** to get one *real* end-to-end eligibility proof on testnet, is the right order: apply DB migration → deploy contracts → close **W2c** (ZK verify) first, deferring TEE/PQC/zkML?
3. **Adapter vs. real backends:** stand up real TEE/EZKL/PQC providers now, or keep them flag-gated for the pilot and prove the ZK + audit + disclosure path first?
4. **Partner sequencing:** Wallet or Cruzible integration first, and do you want ZeroID to own the boundary replication into those repos?
5. **The two failing suites:** fix now or quarantine with a tracked issue?
6. **Security review depth:** what level of external audit (contracts + crypto + key custody) do you want before any mainnet exposure?

Generated 2026-06-29. Numbers are from the live codebase on `feat/economic-flywheel`; test counts are from the most recent local runs (backend full run: 743/746 passing, the 3 failures isolated to the two pre-existing suites above).